Figure 33.2 Disposing of `HttpClient` can lead to socket exhaustion. Each new connection requires the operating system to assign a new socket, and closing a socket doesn't make it available until the `TIME_WAIT` period of 240 seconds has elapsed. Eventually you can run out of sockets, at which point you can't make any outgoing HTTP requests.

Given that I said there are 65,536 different port numbers, you might think that's an unlikely situation. It's true that you will likely run into this problem only on a server that is making a lot of connections, but it's not as rare as you might think.

The problem is that when you dispose of an `HttpClient`, *it doesn't close the socket immediately.* The design of the TCP/IP protocol used for HTTP requests means that after trying to close a connection, the connection moves to a state called `TIME_WAIT`. The connection then waits for a specific period (240 seconds in Windows) before closing the socket.

Until the `TIME_WAIT` period has elapsed, you can't reuse the socket in another `HttpClient` to make HTTP requests. If you're making a lot of requests, that can quickly lead to socket exhaustion, as shown in figure 33.2.

TIP You can view the state of active ports/sockets in Windows and Linux by running the command `netstat` from the command line or a terminal window. Be sure to run `netstat -n` in Windows to skip Domain Name System (DNS) resolution.

Instead of disposing of `HttpClient`, the general advice (before the introduction of `IHttpClientFactory`) was to use a single instance of `HttpClient`, as shown in the following listing.

Listing 33.2 Using a singleton `HttpClient` to avoid socket exhaustion